

Lab 1: Display Density Metrics & Image Array Mechanics in NumPy

Human-Computer Interaction (HCI) & Computer Graphics (CG) — Lab Report

Name: Abdullah | Roll No: 2K24/CSE/19

Institute of Mathematics & Computer Science, University of Sindh, Jamshoro

Deliverables Checklist

- ☐ Documented Python source code (.py / .ipynb) for Tasks 1–4
- ☐ Screenshots of Task 1 output — desktop monitor (1920x1080, 24") and smartphone (1170x2532, 6.1")
- ☐ Matplotlib figure outputs from Task 3 and Task 4
- ☐ Short Answer on RGBA memory overhead

Task 1: Display Pixel Density (PPI/DPI) Calculator

Objective: Write a Python script that calculates screen parameters and classifies display Density based on physical screen attributes.

```
import math

def calculate_screen_metrics():
    # Prompt user for inputs
    w_px = int(input("Enter horizontal resolution (pixels): "))
    h_px = int(input("Enter vertical resolution (pixels): "))
    d_inches = float(input("Enter physical diagonal size (inches): "))

    # 1. Total pixel count
    total_pixels = w_px * h_px

    # 2. Simplified aspect ratio using GCD
    gcd = math.gcd(w_px, h_px)
    aspect_w = w_px // gcd
    aspect_h = h_px // gcd

    # 3. Screen DPI (PPI) calculation
    # Formula:  $\sqrt{W\_px^2 + H\_px^2} / \text{Diagonal\_inches}$ 
    dpi = math.sqrt(w_px**2 + h_px**2) / d_inches

    # 4. Classify display density
    if dpi < 100:
        category = "Low Density (Standard Monitor)"
    elif 100 <= dpi <= 200:
        category = "Medium Density (HD Display)"
    else:
        category = "High Density (Retina / Mobile)"

    # Output formatted results
    print("\n--- DISPLAY METRICS ANALYSIS ---")
    print(f"Total Pixel Count : {total_pixels:,} pixels")
    print(f"Aspect Ratio       : {aspect_w}:{aspect_h}")
    print(f"Calculated DPI      : {dpi:.2f} DPI")
    print(f"Density Category    : {category}")

if __name__ == "__main__":
    calculate_screen_metrics()
```

Screenshot 1 — Desktop Monitor (1920 x 1080, 24")

```
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,073,600 pixels
Aspect Ratio      : 16:9
Calculated DPI    : 91.79 DPI
Density Category  : Low Density (Standard Monitor)
```

Figure 1: Task 1 output for a 1920x1080, 24-inch desktop monitor.

Screenshot 2 — Smartphone Screen (1170 x 2532, 6.1")

```
Enter horizontal resolution (pixels): 1170
Enter vertical resolution (pixels): 2532
Enter physical diagonal size (inches): 6.1

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,962,440 pixels
Aspect Ratio      : 195:422
Calculated DPI    : 457.25 DPI
Density Category  : High Density (Retina / Mobile)
```

Figure 2: Task 1 output for a 1170x2532, 6.1-inch smartphone screen.

Task 2: Environment Setup & Synthetic Image Matrix Creation

Objective: Install Python graphics packages and construct a 3D image matrix from scratch

```
import numpy as np

def create_synthetic_image():
    # 1. Create 300x400x3 matrix initialized with zeros
    height, width, channels = 300, 400, 3
    img = np.zeros((height, width, channels), dtype=np.uint8)

    half_h = height // 2
    half_w = width // 2

    # 2. Fill four quadrants using spatial slicing
    # Top-Left: Red [255, 0, 0]
    img[0:half_h, 0:half_w] = [255, 0, 0]

    # Top-Right: Green [0, 255, 0]
    img[0:half_h, half_w:width] = [0, 255, 0]

    # Bottom-Left: Blue [0, 0, 255]
    img[half_h:height, 0:half_w] = [0, 0, 255]

    # Bottom-Right: White [255, 255, 255]
    img[half_h:height, half_w:width] = [255, 255, 255]

    # 3. Print matrix parameters
    print("--- SYNTHETIC MATRIX METRICS ---")
    print(f"Array Shape (H, W, C) : {img.shape}")
    print(f>Data Type : {img.dtype}")
    print(f"Total Elements : {img.size:,} values")
    print(
        f"Memory Footprint : {img.nbytes:,} bytes ({img.nbytes / 1024:.2f}
KB)"
    )

if __name__ == "__main__":
    create_synthetic_image()
```

Output

```
--- SYNTHETIC MATRIX METRICS ---  
Array Shape (H, W, C) : (300, 400, 3)  
Data Type          : uint8  
Total Elements     : 360,000 values  
Memory Footprint   : 360,000 bytes (351.56 KB)
```

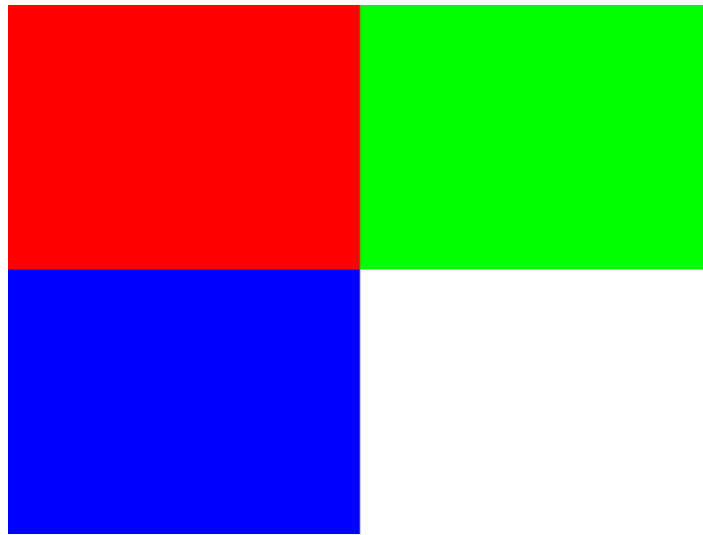


Figure 3: Rendered 300x400 four-quadrant synthetic image (Red / Green / Blue / White).

Task 3: Channel Slicing & Isolation

Objective: Load an image, extract individual 2D channel intensity grids, and isolate single-channel color views.

```
import matplotlib.pyplot as plt  
import numpy as np  
from PIL import Image  
import os  
  
# 1. Load image into a NumPy array  
img_path = "sample.jpg"  
  
# Automatically create a sample image if it doesn't exist  
if not os.path.exists(img_path):
```

```

# Creates a 1080x1920x3 random RGB test image
test_data = np.random.randint(0, 256, (1080, 1920, 3), dtype=np.uint8)
Image.fromarray(test_data).save(img_path)
print(f"Generated a synthetic image at '{img_path}' for testing.")
img = np.array(Image.open(img_path))
# 2. Extract individual 2D intensity grids via Axis 2 slicing
r_2d = img[:, :, 0]
g_2d = img[:, :, 1]
b_2d = img[:, :, 2]
# 3. Construct 3D isolated-channel color images
red_only = np.zeros_like(img)
red_only[:, :, 0] = r_2d

green_only = np.zeros_like(img)
green_only[:, :, 1] = g_2d

blue_only = np.zeros_like(img)
blue_only[:, :, 2] = b_2d
# 4. Console Output Summary
print("--- CHANNEL EXTRACTION SUMMARY ---")
print(f"Original Image Shape : {img.shape}")
print(
    f"Red Channel 2D Shape : {r_2d.shape} | Mean Intensity: {r_2d.mean():.2f}"
)
print(
    f"Green Channel 2D Shape: {g_2d.shape} | Mean Intensity: {g_2d.mean():.2f}"
)
print(
    f"Blue Channel 2D Shape : {b_2d.shape} | Mean Intensity: {b_2d.mean():.2f}"
)
# 5. Display 2x3 Subplot Grid using Matplotlib
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
# Top Row: 3D Color Isolated Views
axes[0, 0].imshow(red_only)
axes[0, 0].set_title("Red-Only Channel (3D)")
axes[0, 0].axis("off")

axes[0, 1].imshow(green_only)
axes[0, 1].set_title("Green-Only Channel (3D)")
axes[0, 1].axis("off")

axes[0, 2].imshow(blue_only)
axes[0, 2].set_title("Blue-Only Channel (3D)")
axes[0, 2].axis("off")

```

```
# Bottom Row: 2D Intensity Grids (Grayscale)
axes[1, 0].imshow(r_2d, cmap="gray")
axes[1, 0].set_title("Red Intensity Grid (2D)")
axes[1, 0].axis("off")

axes[1, 1].imshow(g_2d, cmap="gray")
axes[1, 1].set_title("Green Intensity Grid (2D)")
axes[1, 1].axis("off")

axes[1, 2].imshow(b_2d, cmap="gray")
axes[1, 2].set_title("Blue Intensity Grid (2D)")
axes[1, 2].axis("off")

plt.tight_layout()
plt.show()
print("Display Window : Matplotlib 2x3 Subplot Grid Rendered.")
```

```
--- CHANNEL EXTRACTION SUMMARY ---
Original Image Shape : (768, 1408, 3)
Red Channel 2D Shape : (768, 1408) | Mean Intensity: 131.16
Green Channel 2D Shape: (768, 1408) | Mean Intensity: 128.83
Blue Channel 2D Shape : (768, 1408) | Mean Intensity: 124.34
Display Window : Matplotlib 2x3 Subplot Grid Rendered.
```

Figure 1

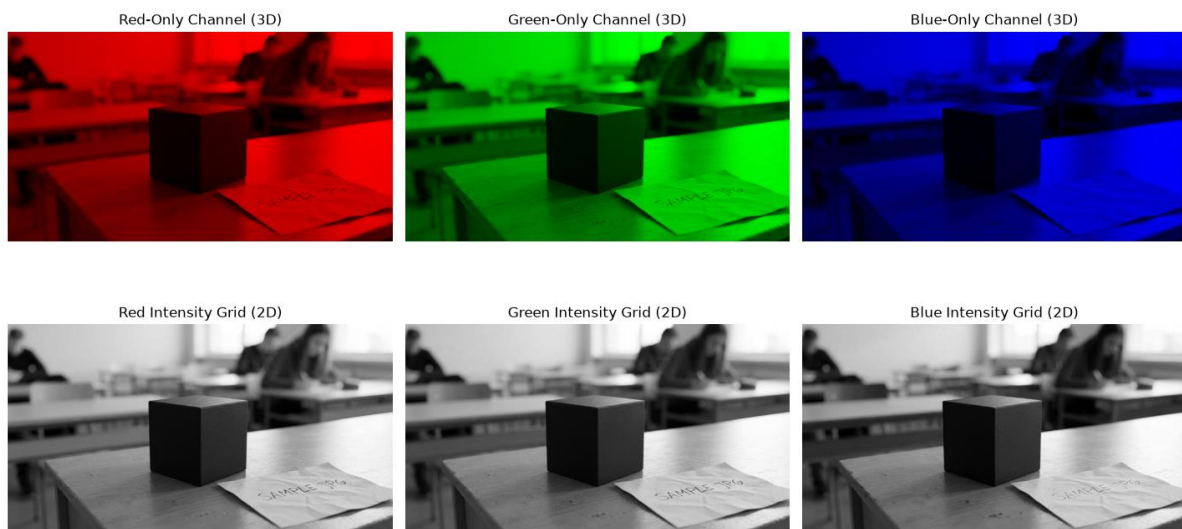


Figure 4: 2x3 subplot - isolated color channels (top) and grayscale intensity maps (bottom).

Task 4: Spatial Downsampling & Pixelation via Striding

Objective: Perform spatial downsampling directly in NumPy using slice steps.

```
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
import os

# 1. Load image into a NumPy array
img_path = "sample.jpg"

# Automatically create a sample image if it doesn't exist
if not os.path.exists(img_path):
    test_data = np.random.randint(0, 256, (1080, 1920, 3), dtype=np.uint8)
    Image.fromarray(test_data).save(img_path)
    print(f"Generated a synthetic image at '{img_path}' for testing.")
img = np.array(Image.open(img_path))

N = 8 # Step factor

# 2. Downsample using NumPy striding
downsampled = img[::N, ::N, :]

# 3. Re-expand back to original dimensions using np.repeat
re_expanded = np.repeat(downsampled, N, axis=0)
re_expanded = np.repeat(re_expanded, N, axis=1)

# Handle cases where original dims aren't perfectly divisible by N
re_expanded = re_expanded[:img.shape[0], :img.shape[1], :]

# 4. Calculate percentage drop in dimensions and memory
original_mem = img.nbytes
downsampled_mem = downsampled.nbytes

dim_reduction = (1 - (downsampled.shape[0] / img.shape[0])) * 100
mem_reduction = (1 - (downsampled_mem / original_mem)) * 100

# Console Output Summary
print(f"--- DOWNSAMPLING ANALYSIS (N = {N}) ---")
print(f"Original Shape      : {img.shape} | Memory: {original_mem:,} bytes")
print(f"Downsampled Shape    : {downsampled.shape} | Memory: {downsampled_mem:,} bytes")
```

```

print(f"Re-expanded Shape   : {re_expanded.shape} | Visual: Blocky Pixelation")
print(f"Dimension Reduction  : {dim_reduction:.2f}% reduction per axis")
print(f"Memory Savings       : {mem_reduction:.2f}% data reduction")

# Display original vs pixelated comparison
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

axes[0].imshow(img)
axes[0].set_title("Original Image")
axes[0].axis("off")

axes[1].imshow(re_expanded)
axes[1].set_title(f"Pixelated (N={N})")
axes[1].axis("off")

plt.tight_layout()
plt.show()

```

```

--- DOWNSAMPLING ANALYSIS (N = 8) ---
Original Shape       : (768, 1408, 3) | Memory: 3,244,032 bytes
Downsampled Shape    : (96, 176, 3) | Memory: 50,688 bytes
Re-expanded Shape    : (768, 1408, 3) | Visual: Blocky Pixelation
Dimension Reduction   : 87.50% reduction per axis
Memory Savings       : 98.44% data reduction

```

Original Image



Pixelated (N=8)



Figure 5: Original image (left) vs. re-expanded pixelated image after N=8 downsampling (right).

Question: why does adding an alpha channel (RGBA) increases an image array's memory consumption by 33%?

Short Answer: Think of an image array like a simple grid of pixels. In a standard RGB image, every single pixel needs 3 pieces of data: red, green, and blue light values.

When you add an Alpha channel (RGBA) to handle transparency, you add a 4th piece of data to every single pixel.

Math behind it:

RGB: 3 values per pixel

RGBA: 4 values per pixel

Going from 3 values to 4 values adds 1 extra value per pixel. Since 1 out of 3 is one-third, your total storage size increases by 33%.

Because computer memory stores pixels in fixed slots, every pixel gets that extra data slot reserved—even if the pixel is completely solid and doesn't actually use any transparency.